

NetGrove Backend — Design Plan

Corporate sustainability challenge & rewards platform — Django + DRF + PostgreSQL

This plan is built on the original NetGrove UML diagram, corrected and extended to support the full product flow: a company admin onboards a company, employees self-register with a company email, managers assign sustainability challenges to departments, employees complete and submit proof, earn points, and redeem those points for brand vouchers (e.g. Adidas, Gucci).

1. Gaps Fixed From the Original UML

Diagram issue	Fix
<code>Rewards</code> and <code>Rewards History</code> are two tables doing the same job (a balance and a ledger)	Merged into <code>PointsWallet</code> (cached balance) + <code>PointsTransaction</code> (append-only ledger). One is the cache, one is the audit trail.
No Voucher/Coupon/Brand model — "convert points into an Adidas/Gucci coupon" has nowhere to live	Added <code>VoucherCatalogItem</code> , <code>VoucherCode</code> , <code>VoucherRedemption</code> — admin-managed voucher catalog per company.
<code>Company</code> has no way to gate which employees can self-register	Added <code>Company.email_domain</code> (e.g. <code>acme.com</code>); employee registration validates the signup email against it.
<code>User</code> has no <code>department</code> field, so department-targeted challenges can't reach anyone	Added <code>User.department</code> FK.
<code>Challenge</code> has no <code>status</code> field even though the <code>ChallengeStatus</code> enum (DRAFT/ACTIVE/COMPLETED) exists disconnected in the diagram	Attached <code>status</code> to <code>Challenge</code> ; also added <code>end_date</code> .
<code>Challenge</code> only links to <code>Company</code> , not <code>Department</code>	Added <code>Challenge.department</code> (nullable — null means company-wide).
<code>Action</code> (proof of completion) has no image/file field and no reviewer	Added <code>proof_image</code> , <code>caption</code> , <code>reviewed_by</code> , <code>reviewed_at</code> .
<code>Action</code> and <code>ChallengeParticipation</code> both independently reference user + challenge	<code>Action</code> becomes a submission under a <code>ChallengeParticipation</code> , not a sibling table.
<code>Notification</code> has no <code>is_read</code> flag	Added.
<code>ActionStatus</code> / participation status only has PENDING/COMPLETED — no rejection path	Extended so managers can reject a submission and send the employee back to retry.

2. Confirmed Decisions

- **Stack:** Django + Django REST Framework + PostgreSQL
- **File storage:** Local disk / server storage (proof photos, logos, profile images) — acceptable for now; revisit before scaling to multiple app servers, since local disk isn't shared/durable across instances
- **Voucher redemption model:** Admin-managed voucher catalog per company — each company's admin defines available vouchers and uploads a pool of redemption codes; employees redeem points for a code, no external brand API integration required

3. Data Model, by Django App

accounts

User (custom model) — email, username, password, first_name, last_name, role (COMPANY_ADMIN / MANAGER / EMPLOYEE), company FK (null until assigned), department FK (null), is_active

Company — name, description, logo, admin FK→User, email_domain, is_active, created_at

Department — company FK, name, manager FK→User (null)

UserProfile — user (1:1), bio, profile_image

challenges

Challenge — company FK, department FK (null = company-wide), created_by FK→User, title, description, challenge_type (WATERSAVE / TREEPLANT / ...), difficulty, point_reward, start_date, end_date, status (DRAFT / ACTIVE / COMPLETED)

ChallengeParticipation — user FK, challenge FK, status (JOINED / SUBMITTED / APPROVED / REJECTED), joined_at, completed_at — unique_together(user, challenge)

Action (submission) — participation FK, action_type, proof_image, caption, status (PENDING / APPROVED / REJECTED), reviewed_by FK→User (null), reviewed_at, submitted_at

rewards

PointsWallet — user (1:1), balance

PointsTransaction — user FK, amount, transaction_type (EARNED / REDEEMED), related_challenge FK (null), related_redemption FK (null), balance_after, created_at

VoucherCatalogItem — company FK, brand_name, title, description, image, points_cost, is_active, created_by

VoucherCode — catalog_item FK, code (unique), is_used, assigned_to FK→User (null), assigned_at

VoucherRedemption — user FK, catalog_item FK, voucher_code (1:1), points_spent, redeemed_at

notifications

Notification — user FK, title, body, notification_type (WELCOME_MESSAGE / CHALLENGE_ASSIGNED / SUBMISSION_APPROVED / SUBMISSION_REJECTED / POINTS_EARNED / VOUCHER_REDEEMED), is_read, created_at

leaderboard

Start as a computed aggregation (query over `PointsTransaction` / `ChallengeParticipation`, scoped by company / department / challenge) rather than a persisted table — avoids keeping a redundant table in sync. Only add a cached `LeaderboardSnapshot` table, recomputed by a periodic task, if a real performance problem shows up later.

4. Core Flows

1. Company onboarding — one transaction creates `User(role=COMPANY_ADMIN, company=None)` + `Company(admin=user, email_domain=...)`, then backfills `user.company`.

2. Employee self-registration — signup email's domain must match `Company.email_domain` → creates `User(role=EMPLOYEE)`, fires a `WELCOME_MESSAGE` notification.

3. Department + manager setup — admin creates a `Department`, assigns a `User` as manager (promoting their role to MANAGER), assigns employees to it.

4. Challenge lifecycle — admin/manager creates a `Challenge` (DRAFT → ACTIVE) scoped to the company or one department → employee sees it in their feed → `join` creates `ChallengeParticipation(JOINED)` → employee uploads proof → `Action(PENDING)`, participation → SUBMITTED → manager approves/rejects → on approve: participation → APPROVED, `PointsTransaction(EARNED)` created, wallet balance updated, `SUBMISSION_APPROVED` notification sent.

5. Redemption — employee browses `VoucherCatalogItem` entries for their company → redeems → atomically checks balance, locks/assigns one unused `VoucherCode`, creates `VoucherRedemption` + `PointsTransaction(REDEEMED)`.

5. Roles & Permissions

Role	Capabilities
COMPANY_ADMIN	Full control within own company — departments, challenges, voucher catalog, promote users to MANAGER, view all employees
MANAGER	Create/edit challenges for their department, review submissions from their department's employees
EMPLOYEE	View/join challenges, submit proof, view own wallet/history, redeem vouchers

Every queryset is scoped by `request.user.company` — enforced via a shared permission/mixin, not per-view checks — so there is no cross-company data leakage.

6. Build Order

1. Django project + DRF + SimpleJWT + PostgreSQL + custom `User` model (must exist before the first migration)
2. `accounts` : models, registration/login, domain-gated employee signup, department management
3. `challenges` : models, create/join/submit/review endpoints
4. `rewards` : wallet + transactions wired via a signal on participation approval; voucher catalog + redemption endpoint
5. `notifications` : model + signal hooks for the events above (simple polling GET for MVP)
6. `leaderboard` : aggregation endpoints
7. Django admin customization scoped per company (free win for admins managing voucher codes/challenges)
8. `drf-spectacular` for OpenAPI docs, seed fixtures, tests